

NATIONAL UNIVERSITY OF SCIENCES & TECHNOLOGY

School of Electrical Engineering & Computer Science (SEECS)

Parallel and Distributed Processing | HPC Project

HPC Setup & Execution

Laplace Solver

Course	Parallel and Distributed Processing
Designed by	Awais Asghar
Instructor	Dr. Attique Dawood
HPC System	NUST RCMS Supercomputer
Compute Node	compute-0-5
Date	May 2026

Overview

This document is a complete guide covering every single step required to log in, upload, compile, run, and verify the Laplace Solver project on the NUST RCMS HPC Supercomputer. Every command is written exactly as it should be typed. Nothing is skipped.

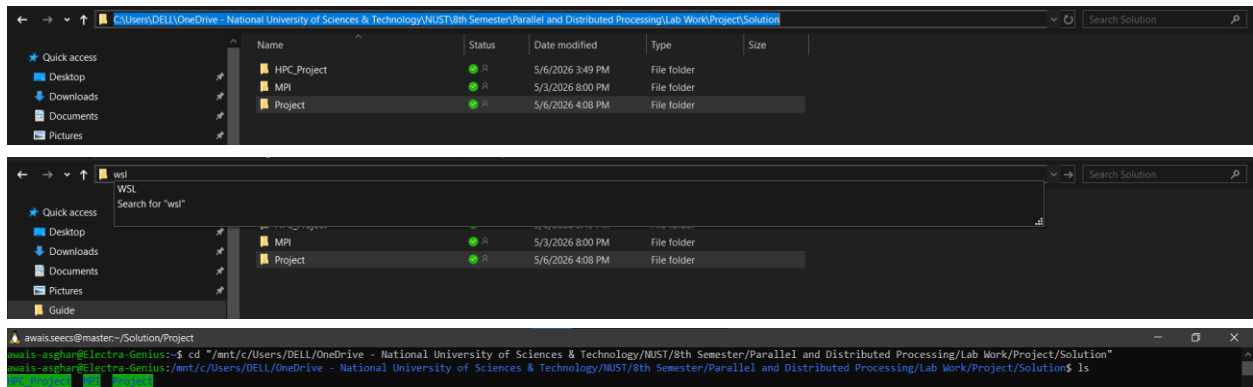
What you need	PC with WSL (Ubuntu) or a Linux machine
HPC username	awais.seecs (replace with your own username)
HPC IP address	10.19.10.50
Head node name	afrit
Compute node used	compute-0-5
Project folder	C:\Users\DELL\OneDrive - National University of Sciences & Technology\NUST\8th Semester\Parallel and Distributed Processing\Lab Work\Project\Solution\Project
Offline nodes	compute-0-7, 14, 15, 20, 25, 29, 30 (do not use)

WARNING: Never run simulations on the head node (afrit). Always SSH into a compute node first. Any simulation on the head node will be terminated without warning and data may be lost.

Phase 1: Uploading the Project to HPC

1.1 Open Your Terminal

Open WSL (Windows Subsystem for Linux) on your Windows PC in the Project Folder, or a Linux terminal on a Linux machine. All commands in this document are typed in this terminal window.



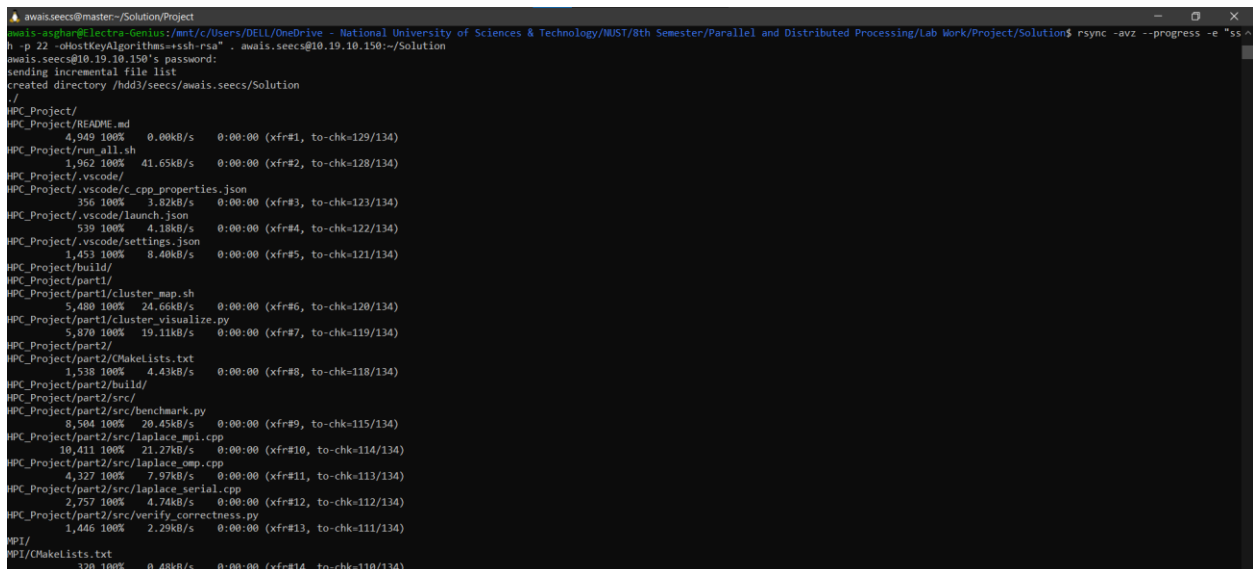
1.2 Upload the Project Folder to HPC

Run this one command. There are 2 commands for temporary users use scp and for your own username use 2nd command.

```
scp -r -O -oHostKeyAlgorithms=ssh-rsa Project user2@10.19.10.50:~
```

```
rsync -avz --progress -e "ssh -p 22 -oHostKeyAlgorithms=+ssh-rsa" .  
awais.seecs@10.19.10.150:~/Solution
```

Enter your HPC password when prompted. You will see each file uploading with progress percentages:



Phase 2: Again Logging Into the HPC

2.1 SSH Into the HPC Head Node

Type one of these commands according to your user type exactly to connect to the HPC. Confirm the content in your HPC using **ls**.

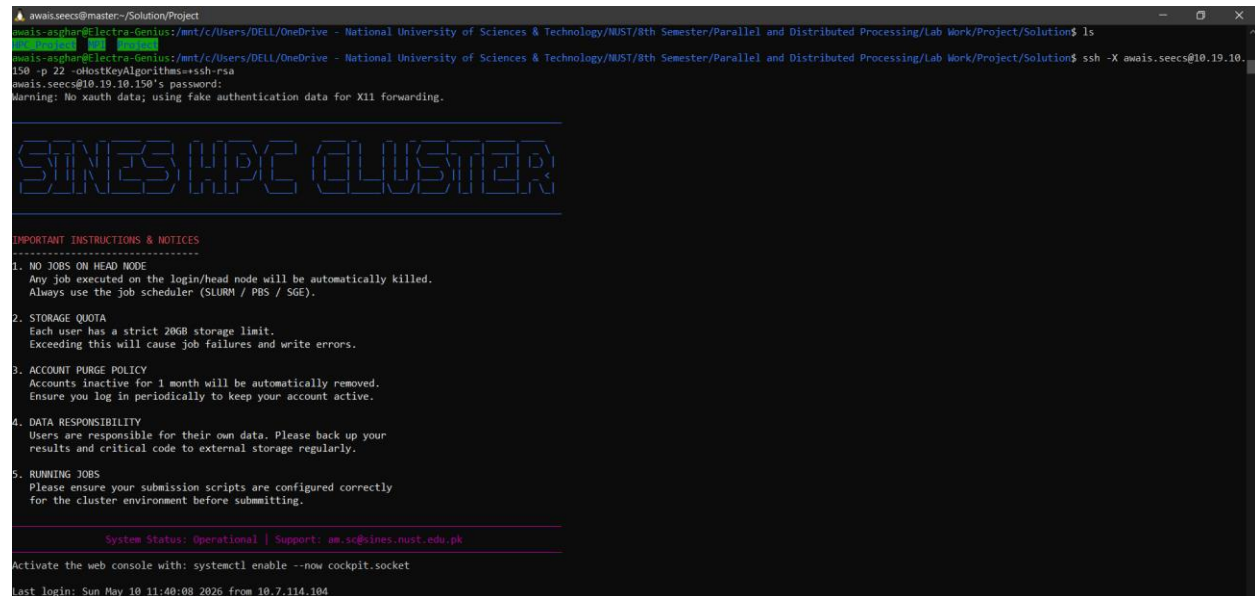
Default Username:

```
ssh -X user2@10.19.10.50 -oHostKeyAlgorithms+=ssh-rsa  
ls
```

Personal Username:

```
ssh -X awais.seecs@10.19.10.150 -p 22 -oHostKeyAlgorithms+=ssh-rsa  
ls
```

I am using my personal Username:



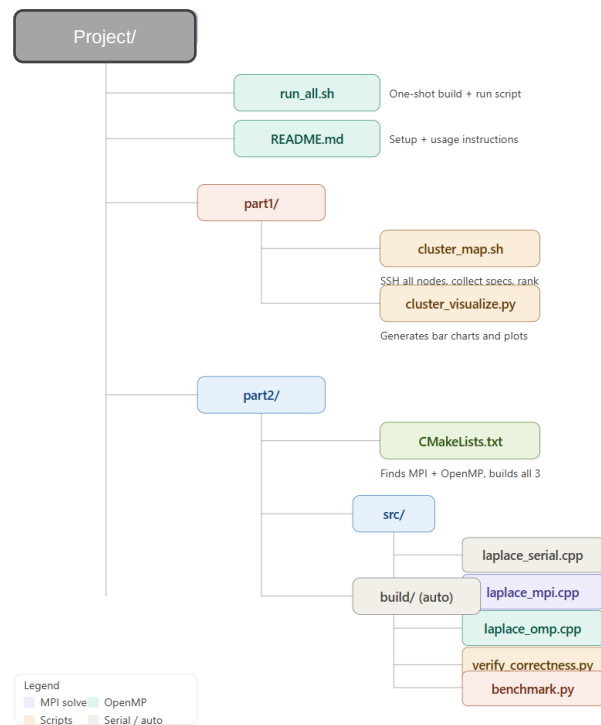
The terminal window shows the user 'awais.seecs' logging into the HPC head node. The banner displays 'SINES HPC CLUSTER' in a stylized font. Below the banner, there are 'IMPORTANT INSTRUCTIONS & NOTICES' listed in five points. At the bottom, the system status is 'Operational' and the support email is 'aw.sc@sines.nust.edu.pk'.

```
awais.seecs@master-~/Solution/Project  
awais-asphar@electra-genius:/mnt/c/Users/DELL/OneDrive - National University of Sciences & Technology/NUST/8th Semester/Parallel and Distributed Processing/Lab Work/Project/Solution$ ls  
awais-asphar@electra-genius:/mnt/c/Users/DELL/OneDrive - National University of Sciences & Technology/NUST/8th Semester/Parallel and Distributed Processing/Lab Work/Project/Solution$ ssh -X awais.seecs@10.19.10.150 -p 22 -oHostKeyAlgorithms+=ssh-rsa  
awais.seecs@10.19.10.150's password:  
Warning: No xauth data; using fake authentication data for X11 forwarding.  
  
SINES HPC CLUSTER  
  
-----  
IMPORTANT INSTRUCTIONS & NOTICES  
-----  
1. NO JOBS ON HEAD NODE  
Any job executed on the login/head node will be automatically killed.  
Always use the job scheduler (SLURM / PBS / SGE).  
  
2. STORAGE QUOTA  
Each user has a strict 20GB storage limit.  
Exceeding this will cause job failures and write errors.  
  
3. ACCOUNT PURGE POLICY  
Accounts inactive for 1 month will be automatically removed.  
Ensure you log in periodically to keep your account active.  
  
4. DATA RESPONSIBILITY  
Users are responsible for their own data. Please back up your  
results and critical code to external storage regularly.  
  
5. RUNNING JOBS  
Please ensure your submission scripts are configured correctly  
for the cluster environment before submitting.  
  
-----  
System Status: Operational | Support: aw.sc@sines.nust.edu.pk  
-----  
Activate the web console with: systemctl enable --now cockpit.socket  
Last login: Sun May 10 11:40:08 2026 from 10.7.114.104
```

2.2 Navigate to the Build Folder

```
cd Solution  
ls  
  
cd Project  
ls  
  
cd part2  
ls  
  
cd build  
ls
```

Project Folder Hierarchy:



```
[awais.seecs@master ~]$ ls
Solution
[awais.seecs@master ~]$ cd Solution
[awais.seecs@master Solution]$ ls
HPC_Project MPI_Project
[awais.seecs@master Solution]$ cd Project
[awais.seecs@master Project]$ ls
README.md build part1 part2 run_all.sh
[awais.seecs@master Project]$ cd part2
[awais.seecs@master part2]$ ls
CMakeLists.txt build src
[awais.seecs@master part2]$ cd build
[awais.seecs@master build]$ ls
[awais.seecs@master build]$
```

2.3 Build the project using cmake

```
cmake ..
```

```
[awais.seecs@master build]$ cmake ..
CMake Deprecation Warning at CMakeLists.txt:1 (cmake_minimum_required):
  Compatibility with CMake < 2.8.12 will be removed from a future version of
  CMake.

  Update the VERSION argument <min> value or use a ...<max> suffix to tell
  CMake that the project does not need compatibility with older versions.

-- The CXX compiler identification is GNU 8.5.0
-- Detecting CXX compiler ABI info
-- Detecting CXX compiler ABI info - done
-- Check for working CXX compiler: /usr/bin/c++ - skipped
-- Detecting CXX compile features
-- Detecting CXX compile features - done
-- Found MPI_CXX: /usr/lib64/openmpi/lib/libmpi_cxx.so (found version "3.1")
-- Found MPI: TRUE (found version "3.1")
-- MPI found: /usr/lib64/openmpi/bin/mpicxx
-- Found OpenMP_CXX: -fopenmp (found version "4.5")
-- Found OpenMP: TRUE (found version "4.5")
-- OpenMP found
--
-- Build complete. Usage:
--   Serial : ./laplace_serial [N] [epochs] [tol]
--   MPI    : mpirun -n 4 ./laplace_mpi [N] [epochs] [tol]
--   OpenMP : ./laplace_omp [N] [epochs] [tol] [threads]
--   Verify : python3 ../src/verify_correctness.py [N]
--   Bench  : python3 ../src/benchmark.py
--
-- Configuring done (3.7s)
-- Generating done (0.0s)
-- Build files have been written to: /hdd3/seecs/awais.seecs/Solution/Project/part2/build
[awais.seecs@master build]$
```

2.4 Compile All Three Solvers

```
make -j4
```

```
[awais.seecs@master build]$ make -j4
[ 16%] Building CXX object CMakeFiles/laplace_mpi.dir/src/laplace_mpi.cpp.o
[ 33%] Building CXX object CMakeFiles/laplace_serial.dir/src/laplace_serial.cpp.o
[ 50%] Building CXX object CMakeFiles/laplace_omp.dir/src/laplace_omp.cpp.o
[ 66%] Linking CXX executable laplace_serial
[ 83%] Linking CXX executable laplace_omp
[ 83%] Built target laplace_serial
[ 83%] Built target laplace_omp
[100%] Linking CXX executable laplace_mpi
[100%] Built target laplace_mpi
[awais.seecs@master build]$
```

2.5 Verify Binaries Were Created

```
ls
```

```
[awais.seecs@master build]$ ls
CMakeCache.txt  CMakeFiles  Makefile  cmake_install.cmake  laplace_mpi  laplace_omp  laplace_serial
[awais.seecs@master build]$
```

Phase 3: Setting Up the Results Folder

3.1 Create the Results Directory

```
mkdir -p ~/Project/part2/results
```

3.2 Copy Binaries and Scripts Into Results

Copy all compiled binaries and Python scripts into the results folder so everything is available in one place on the compute node:

```
cp ./laplace_mpi ~/Project/part2/results/  
cp ./laplace_omp ~/Project/part2/results/  
cp ./laplace_serial ~/Project/part2/results/  
cp ../src/verify_correctness.py ~/Project/part2/results/
```

```
[awais.seecs@master build]$ cp ./laplace_mpi ~/Project/part2/results/  
lace_omp[awais.seecs@master build]$ cp ./laplace_omp ~/Project/part2/results/  
rial ~/Project/part2/results/  
cp ../src/verify_correctness.py[awais.seecs@master build]$ cp ./laplace_serial ~/Project/part2/results/  
[awais.seecs@master build]$ cp ../src/verify_correctness.py ~/Project/part2/results/  
[awais.seecs@master build]$ cp ../src/benchmark.py ~/Project/part2/results/
```

3.3 Confirm Everything Is in Place

```
cd ..  
ls -lh ~/Project/part2/results/
```

```
[awais.seecs@master build]$ ls -lh ~/Project/part2/results/  
total 180K  
-rwxr-xr-x 1 awais.seecs seecs 8.4K May 10 12:25 benchmark.py  
-rwxr-xr-x 1 awais.seecs seecs 103K May 10 12:25 laplace_mpi  
-rwxr-xr-x 1 awais.seecs seecs 29K May 10 12:25 laplace_omp  
-rwxr-xr-x 1 awais.seecs seecs 28K May 10 12:25 laplace_serial  
-rwxr-xr-x 1 awais.seecs seecs 1.6K May 10 12:25 verify_correctness.py
```

3.4 SSH Into the Compute Node

Go to result folder and check the that files (benchmark.py laplace_mpi laplace_omp laplace_serial verify_correctness.py) are visible or not. Then run the serial solver and MPI solver with 4 processes. Run the OpenMP solver using with 4 threads and run the verification script. **Please check the python version on your HPC.** In my case, I have a python3.

```
cd ~/Project/part2/results
./laplace_serial 32 10000 1e-4
mpirun -n 4 ./laplace_mpi 32 10000 1e-4
./laplace_omp 32 10000 1e-4 4
python3 verify_correctness.py 32
```

Phase 4: Running the Full Benchmark

Run each command one at a time. Wait for each to finish before typing the next one.

4.1 MPI Scaling Test (N=256, Varying Processes)

Tests how MPI speedup grows as we add more processes:

```
mpirun -n 1 ./laplace_mpi 256 5000 1e-4
```

```
mpirun -n 2 ./laplace_mpi 256 5000 1e-4
```

```
mpirun -n 4 ./laplace_mpi 256 5000 1e-4
```

```
mpirun -n 8 ./laplace_mpi 256 5000 1e-4
```

4.2 OpenMP Scaling Test (N=256, Varying Threads)

```
./laplace_omp 256 5000 1e-4 1
```

```
./laplace_omp 256 5000 1e-4 2
```

```
./laplace_omp 256 5000 1e-4 4
```

```
./laplace_omp 256 5000 1e-4 8
```

4.3 Serial Baseline for Different Grid Sizes

```
./laplace_serial 256 5000 1e-4
```

```
./laplace_serial 512 5000 1e-4
```

4.4 Grid Size Scaling With 4 Procs and 4 Threads

```
mpirun -n 4 ./laplace_mpi 128 5000 1e-4
```

```
mpirun -n 4 ./laplace_mpi 256 5000 1e-4
```

```
mpirun -n 4 ./laplace_mpi 512 5000 1e-4
```

```
./laplace_omp 128 5000 1e-4 4
```

```
./laplace_omp 256 5000 1e-4 4
```

```
./laplace_omp 512 5000 1e-4 4
```

Phase 5: Generating Graphs

5.1 Print the content of csv files

```
cat mpi_timing.csv
```

```
cat omp_timing.csv
```

```
cat serial_timing.csv
```

5.3 Generate All Graphs on Your PC

Install matplotlib if not already installed. Use Matplotlib to generate the png files of graphs.

```
pip install matplotlib numpy
```

Graphs you now have for the report:

speedup_comparison.png

execution_time.png

grid_scaling.png

efficiency.png

bar_comparison.png

mpi_scaling.png

Phase 6: Part I - Mapping the HPC Cluster

Part I of the project requires mapping all compute nodes, finding CPU specs, identifying busy nodes, and ranking them by availability.

6.1 SSH Back Into HPC and Go to Part 1 Folder

```
ssh -X awais.seecs@10.19.10.150 -p 22 -oHostKeyAlgorithms+=ssh-rsa  
cd ~/Project/part1
```

6.2 Run the Cluster Mapping Script

This script SSHes into every compute node from compute-0-0 to compute-0-28, collects CPU model, core count, RAM usage, and system load, then ranks all nodes by availability score:

```
bash cluster_map.sh
```

This takes 2 to 4 minutes as it probes 29 nodes. Each node is checked in real time. When finished two files are created:

- cluster_report.txt - human readable ranking table
- cluster_data.csv - comma separated data for graph generation

6.3 View the Node Ranking

```
cat cluster_report.txt
```

6.4 Generate Cluster Graphs

On your PC, download the CSV and generate graphs:

node_availability.png

node_ram.png

node_cpu_specs.png

numa_architecture.png

ram_pie.png